

タクトーン 質疑応答チートシート

チーム 23 — Arduino オーケストラ

2026-07-15 成果発表会 用

1 システム全体像

1.1 構成

ノード	ハードウェア	役割
指揮者 (node_01)	XIAO ESP32-S3 Sense + GY-521 (MPU6050)	IMU ジェスチャ認識, SoftAP 起動, UDP で CTRL/BEAT 配信
楽器 (node_02-05)	Arduino UNO R4 WiFi ×4	WiFi STA 接続, 時計同期, 楽譜進行, USB Serial で NOTE を PC へ送信
ドラム (node_06)	Arduino UNO R4 WiFi ×1	同上 (ドラム譜)
PC (各楽器に 1 台)	Mac + Processing 4	Serial 受信 → 加算合成で発音, ゲーム UI

1.2 データフロー

- 指揮者が IMU (5ms 周期) でジェスチャを取得し、振り下ろしから拍を検出
- 拍検出ごとに BEAT パケット (4 連送) を UDP マルチキャスト (239.0.0.1:5001) で全楽器へ配信
- CTRL パケットを 20 Hz (50ms 周期) で常時配信 (BPM, velocity, 状態, ゲーム情報)
- 楽器ノードは CTRL の timestampMs で時計同期し、playAtMasterMs の到達時刻で発音を予約
- 発音時刻に楽譜から NOTE パケット (20 B) を組み立て、USB Serial (115200 bps) で PC へ送信
- Processing が NOTE を受信し、JSON 楽器定義に基づく加算合成で発音

1.3 パケット仕様 (共通 20 バイト固定)

種別	type	方向	ペイロード概要
CTRL	1	指揮者 → 全楽器 (UDP)	bpmQ8 (BPM×8), velocity, state, mode, navCursor, targetBpm, score
BEAT	2	指揮者 → 全楽器 (UDP)	beatNo, playAtMasterMs (発音予約時刻)
NOTE	3	楽器 → PC (USB Serial)	partId, noteNumber, velocity, gate, durationMs, instrumentId
UI	4	楽器 → PC (USB Serial)	state, mode, navCursor, targetBpm, score, partId, bpmQ8

ヘッダ: magic 0x4F52 (ASCII “OR”), version 0x01, type, seq (単調増加), timestampMs (マスタ時刻) = 12 B. ペイロード 8 B. リトルエンディアン. `static_assert` で 20 B を保証.

2 Arduino — WiFi / UDP 通信

実装要点

- **トポロジ**: 指揮者が SoftAP (SSID OrchestraAP, パスワード orchestra2026, ch6) を起動. 楽器は STA で接続
- **プロトコル**: UDP マルチキャスト 239.0.0.1:5001. TCP は使わない (低遅延・ブロードキャスト不要のため)
- **BEAT 連送**: 同一 BEAT を **4 連送** (beatRedundancy=4), 各回 **2 ms** 間隔 (beatGapMs=2) で送信. radio 状態を変化させてまとめ落ちを軽減
- **CTRL 送信**: 20 Hz (50 ms 間隔). BEAT より後に送信 (playAtMasterMs のマージンが薄いため BEAT 優先)
- **発音予約**: playAtMasterMs = masterNow + 220 ms. SoftAP の DTIM バッファリングによるバースト配送 (~204.8 ms 周期) を吸収するための余裕
- **時計同期**: 楽器は CTRL の timestampMs を使い min フィルタ (窓長 2000 ms) で offset を推定. 窓内の最大 offset サンプル = 最小配送遅延のサンプルを採用. スナップ閾値 1000 ms で指揮者リセットに即追従
- **WiFi 再接続**: STA 側は isLinkUp() が false になると 2000 ms 間隔で自動再接続. 復帰時に udp.stop() → beginMulticast() でマルチキャスト購読を貼り直す
- **ライブラリ**: ESP32 は WiFi.h + WiFiUdp.h, UNO R4 は WiFiS3.h + WiFiUdp.h (#if defined(ARDUINO_ARCH_ESP32) で分岐)

Q. なぜ TCP ではなく UDP マルチキャストを選んだのか？

A. 1 台の指揮者から複数台の楽器へ同時配信するため. TCP はコネクション管理のオーバーヘッドと再送遅延があり, リアルタイム音楽同期には不向き. UDP はパケットロスの可能性はあるが, BEAT 4 連送で冗長化している.

Q. パケットロスはどう対処している？

A. BEAT は 4 連送 (2 ms 間隔) で radio 状態を変えながら送り, まとめ落ちを軽減. 楽器側は同一 beatNo の重複到着を検出し, 初到着分だけ発音予約に使い, 2 個目以降は時計同期の追加サンプルとして利用.

Q. 時計同期の方式は NTP ベースか？

A. NTP は使っていない. CTRL パケットの timestampMs (指揮者の millis() 値) と楽器の millis() の差分 (offset) を min フィルタで推定する独自方式. NTP の往復計測は不要で, 片方向の配信だけで同期できる. min フィルタは窓内の最大 offset (= 配送遅延が最小だったサンプル) を採用することで, バッファリング遅延の影響を排除する.

Q. 220 ms の遅延は体感で問題にならないか？

A. 120 BPM で約半拍の固定遅延. 全楽器に同一の遅延がかかるため, 楽器同士の同期は保たれる. 指揮者の振りと発音の間に一定のずれがあるが, 実測で聴覚上の違和感は報告されていない. ビーコン間隔の最適化で 70 ms 程度まで短縮できる見込み.

Q. SoftAP のチャンネルは固定か？

A. ch6 固定. 2.4 GHz の中間チャンネルで干渉が比較的少ない. 本番環境 (教室) では他の AP との干渉があり得るが, SoftAP なので周囲の AP に合わせたチャンネル選択は行っていない.

Q. 同時接続台数の上限は？

A. ESP32 SoftAP は最大 6 台 (設定値). 楽器 5 台で制限内.

3 Arduino — IMU モーション検出

実装要点

- **センサ:** GY-521 (MPU6050), I2C 接続 (アドレス 0x68), ピン D4(GPIO5)=SDA, D5(GPIO6)=SCL
- **設定:** 加速度 $\pm 4\text{ g}$ (感度 8192 LSB/g), ジャイロ $\pm 2000\text{ dps}$ (16.4 LSB/dps)
- **サンプリング:** 5 ms 周期 (200 Hz), 14 バイトバースト読み取り (加速度 3 軸 + 温度 + ジャイロ 3 軸)
- **LPF:** IIR ローパスフィルタ $\alpha = 0.10$ (時定数 $\approx 45\text{ ms}$)
- **動加速度:** $\text{dynNorm} = |\vec{a}_{\text{LPF}}| - g_{\text{calib}}$. キャリブレーション時の静止加速度ノルム ($\approx 1\text{ g}$) を引くことで姿勢非依存
- **キャリブレーション:** 起動後 WiFi 接続完了で Calibrating に遷移. 2 秒間 (400 サンプル) の加速度ノルム平均を静止ノルムとして保持

3.1 拍検出アルゴリズム (ゲート式ステートマシン + 早期発火)

1. **Idle:** $\text{dynNorm} \leq 1.20\text{ g}$ で待機. 積分は走らせない (ノイズ蓄積防止)
2. **Armed 突入:** $\text{dynNorm} > 1.20\text{ g}$ で Armed に遷移. 動加速度を二重積分して擬似経路長 sPathLen を計算開始
3. **早期発火:** $\text{sPathLen} \geq 0.20\text{ m}$ に到達した瞬間に拍を発火 (振り始めから $\sim 100\text{ ms}$). 1 Armed セッション中 1 回限り. 不応期 350 ms (上限 $\approx 170\text{ BPM}$)
4. **発火後も Armed 維持:** Idle に戻さないことで「発火 $\rightarrow$ 即 Idle $\rightarrow$ 再 Arm $\rightarrow$ 再発火」の連続発火ループを構造的に防止
5. **リリース判定 (デバウンス 40 ms):**
 - (a) $\text{dynNorm} < 0.20\text{ g}$ (絶対値: 完全停止)
 - (b) $\text{dynNorm} < \text{ピーク} \times 0.40$ (相対値: ピークアウト後の減衰)
 - (c) Armed 開始から 800 ms 経過 (タイムアウト保険)
6. **テンポ推定:** 拍間隔の EMA ($\alpha = 0.30$), 範囲 40–240 BPM にクランプ

Q. なぜ加速度のノルムで判定するのか? 軸指定ではダメか?

A. 軸指定だとセンサの取り付け角度で誤検出・取りこぼしが起きた. ノルムは姿勢に依存しないスカラー量なので, どの向きで振っても安定して検出できる. ただし重力に直交する振りは過小評価される ($\sqrt{1+a^2} - 1 < a$) という限界はある.

Q. 1.20 g の閾値はどう決めたか?

A. 仕様書の 1.8 g は重力込み前提で届かず, 0.8 g では小さい揺れで誤検出した. 実機で調整して中間の 1.2 g に決定. 取りこぼし・誤検出が出たら 1.0–1.4 の範囲で再調整する想定.

Q. なぜ早期発火 (振り始めで発火) を採用したか?

A. 振り終わり (リリース時) に発火すると体感遅延が大きく, 指揮者の振りと言がずれる. 擬似経路長 (動加速度の二重積分) が 0.20 m に達した時点で発火すると, 振り始めから $\sim 100\text{ ms}$ で音が出て体感的に同期する.

Q. 連続スイングで二重発火しないか?

A. 構造的に防止している. (1) 発火しても Armed を維持し Idle に戻さないで, 同じ振り中に再 Arm $\rightarrow$ 再発火するループが起きない. (2) 不応期 350 ms で物理的に速い連打も抑制. (3) リリース判定にデバウンス 40 ms を設けて一瞬の dynNorm の落ち込みによる誤リリースも防止.

Q. ジャイロは使っているか?

A. 加速度のみで拍検出. ジャイロデータは取得しているが, 現在のロジックでは判定に使っていない. メニューナビ

(横/縦の振り分け) は加速度ベースの重力推定 + 振り方向分解で実現している。

4 Processing — UI とシリアル通信

実装要点

- **フレームワーク**: Processing 4 + Minim ライブラリ (加算合成オーディオ)
- **描画フレームレート**: 90 fps (`frameRate(90)`)
- **オーディオ**: ステレオ出力, サンプルレート 44100 Hz, バッファ 512 サンプル
- **シリアル**: USB Serial 115200 bps. `serialEvent()` コールバック (Serial スレッド) でバイト受信 → magic バイト走査でフレーミング → 20 B パケットを `ConcurrentLinkedQueue` に push
- **パケット処理**: draw スレッドで毎フレーム `drainPackets()` → NOTE (type=3) なら発音トリガ, UI (type=4) なら画面状態を更新
- **フレーミング**: magic バイト 0x52 0x4F を走査し 2 バイト一致で `inFrame=true`, 以降 18 バイト読んで 20 B パケット完成
- **役割自動判定**: UI パケット (type=4) を受信 → メイン操作 UI, NOTE のみ + `partId ≠ 0x02` → アナライザ
- **マスターリセット検知**: UI パケットが 2000 ms 途絶えたら `uiState=ST_IDLE` に戻し全発音停止

4.1 画面構成

画面	表示内容
PortSelect	シリアルポート一覧 (USB フィルタ+スクロール対応, クリックで開閉)
Waiting	キャリブレーション中 / 待機中 / Fallback
Menu	モード選択 (自由演奏 / ゲーム), カーソル表示
FreePlay	演奏画面 (BPM 表示, パート別最終ノート, ログパネル)
GamePlay	ゲーム演奏画面 + メトロノームガイド (フェードアウトする視覚+音声ガイド)
Result	ゲーム得点表示 (0-100 点)
Analyzer	アナライザ画面 (パート別最終ノート表示)

Q. なぜ Processing を採用したのか？

A. Arduino UNO R4 WiFi 単体では音声合成の処理能力が足りない (44.1 kHz サンプリングの加算合成は浮動小数点演算が大量に必要). Processing は Java ベースで Minim ライブラリによる UGen ベースのリアルタイム音声合成が容易で, Serial ライブラリも標準搭載されている。

Q. シリアル受信でデータが壊れたらどうなる？

A. magic バイト走査 (0x52 0x4F) で毎回先頭を探索するため, 途中のバイト欠損・ノイズが入っても次の magic から自動復帰する. version チェック (0x01) も行い, 無効パケットは破棄する。

Q. なぜ ConcurrentLinkedQueue を使っているか？

A. Serial スレッド (Processing の `serialEvent`) と draw スレッドが別スレッドで動くため. ロックフリーのキューで競合を避け, 描画スレッドが毎フレーム drain する構造で安全にパケットを受け渡す。

Q. 複数ポートを同時に開ける理由は？

A. 楽器は 5 台 + ドラム 1 台で, 各 PC が 1 台を担当する運用だが, デバッグ時には複数台を 1 台の PC に繋いでテストできる. ポートごとに `PortConn` オブジェクトで受信バッファを独立管理している。

Q. frameRate 90 は高すぎないか？

A. パケット受信は Serial スレッドなので frameRate は描画と音声管理の更新頻度. 90fps にすることで drainPackets() の呼び出し間隔を ~11 ms に抑え, パケット滞留を減らしている. 音声処理の発音タイミングにも影響する.

5 音声解析・音色合成

実装要点

- 合成方式: 加算合成 (Additive Synthesis). 各楽器音を倍音の正弦波の重ね合わせで生成
- ピッチ: MIDI ノート番号から基本周波数を算出: $f_0 = 440 \times 2^{(n-69)/12}$
- 倍音: 各倍音の周波数 $f_k = f_0 \times r_k \times \sqrt{1 + B \cdot k^2}$. r_k は倍音比 (理想では k), B は非調和性係数 (inharmonicity)
- ビブラート: ピッチ変調 $\text{pitchMul} = 2^{(d \times g \times \sin \phi_{\text{vib}})/1200}$ (cent 単位). onset 遅延あり
- トレモロ: 振幅変調 $s \times (1 - d/2 + d/2 \cdot \sin \phi_{\text{trem}})$
- エンベロープ: ADSR (attack/decay/sustain/release) + JSON 定義のサンプルカーブ (sustaining 時はループ区間で反復)
- ノイズ: 事前生成のノイズテーブルをエンベロープで変調し加算 (アタックノイズの再現)
- 音色定義: pc_app/production/orchestra_resynth/data/*.instrument.json. ファイル名昇順の index が instrumentId
- ドラム: 別エンジン (DrumEngine.pde) で打楽器専用の合成
- 同時発音: 最大 24 (MAX_POLYPHONY=24), 超過時は最古のボイスを強制リリース
- 消音: Processing 側が durationMs から自動消音. NoteOff は送らない (NoteOn のみ)

5.1 輪唱の実現方法

- 全楽器ノードが同一の楽譜 (score_data.cpp) を持つ
- 声部ごとに headRestBeats を設定: node_02=0, node_03=8, node_04=16, node_05=24
- 楽譜の進行は「指揮者の拍番号 - 1 - headRestBeats」で index を算出
- headRestBeats 分の拍を休符として読み飛ばし, 声部ごとにフレーズ単位でずれて入る
- 周回は CANON_CYCLE_BEATS=56 (曲長 32 + 最終声部遅延 24) のサイクル窓で管理
- 曲: かえるのうた (C major, 32 拍 = 4 フレーズ). 4 声部が 8 拍ずつずれて重なる

Q. なぜ加算合成を選んだのか？

A. 楽器の音色を物理的な特性 (倍音構造, 非調和性, エンベロープ) から再合成できるため. サンプリング方式と違い, テンポに依存する音長やピッチの自由な制御が可能. JSON で音色パラメータを外部定義でき, 楽器変更もファイル差し替えて済む.

Q. 音色の JSON はどこから来たか？

A. sound_lab/ の analyzer.py で実際の楽器音の録音から分析・抽出した. 基音検出 (pyin), 倍音抽出 (FFT/STFT), ADSR フィッティング, 残差ノイズ分離の 9 段パイプラインで JSON を自動生成.

Q. 音階の精度はどの程度か？

A. MOP2 検証 (合成音の基音推定 vs 平均律) で全 24 音の平均 | 誤差 | 0.74 cent, 最大 1.91 cent. 基準 3.6 cent 未満を達成. 誤差の主因は harmonics[n=1].ratio $\neq$ 1.0 (ホルン -1.91, トランペット -0.87 cent).

Q. 同時発音 24 で足りるか？

A. 4 声部 + ドラムが同時に鳴る最大ケースでも 5-6 音程度. durationMs 自動消音で不要なボイスは即リリースされ

るため、24 は十分な余裕がある。

Q. 非調和性 (inharmonicity) とは？

A. 実際の楽器の倍音は理論整数倍からずれる。ピアノの弦の剛性が典型例で $f_k = k \cdot f_0 \cdot \sqrt{1 + B \cdot k^2}$ で高次倍音ほど高くずれる。金管楽器は B が非常に小さい (ほぼ調和的)。

6 ログ取得・デバッグ

実装要点

- **ファーム側:** SerialDebug.h マクロ (DBG_PRINTF / DBG_PRINTLN). SERIAL_DEBUG ビルドフラグで ON/OFF
 - 指揮者: SERIAL_DEBUG=1 (拍検出のデバッグ情報を 200ms ごとにダンプ)
 - 楽器: SERIAL_DEBUG=0 (USB Serial は NOTE バイナリパケット専用. テキスト混在を避ける)
- **指揮者ダンプ:** 200ms 周期で時刻・状態・WiFi・IMU 値・dynNorm・ゲート状態・経路長・BPM・beatNo・シーケンス番号を 1 行出力
- **イベントログ:** 状態遷移・WiFi 接続変化・拍検出の発生時に即時出力 (dumpEdges)
- **Processing 側:** 構造化ログ (OrcLogger.pde): [HH:MM:SS.mmm] [ノード ID] [カテゴリ] メッセージ 形式で stdout + 画面リングバッファ (200 行)
 - カテゴリ: NOTE, UI, SERIAL, AUDIO, STATE, ERROR, SYSTEM, METRO
- **MOP テスト:** MopTest.h で MOP_TEST ビルドフラグ切替. MOP1 (拍検出精度), MOP4 (同期誤差), MOP5 (通信遅延), MOP7 (起動時間), MOP8 (CPU 使用率) を個別に計測出力
- **シリアルログ収集:** tools/verification/scripts/serial_logger.py で複数ポートを同時に開き, PC 側タイムスタンプ + ポート名を付与して保存. pyserial 使用
- **MOP 分析:** tools/verification/scripts/ 配下の Python スクリプト群でログを解析 → PASS/FAIL 判定 → グラフ PNG 出力

Q. デバッグ出力の ON/OFF はどう切り替えるか？

A. platformio.ini の build_flags に -DSERIAL_DEBUG=1 を設定する。0 にするとマクロが空展開になり、コードサイズも実行時コストも 0 になる。楽器ノードは NOTE バイナリを Serial に流すため、デバッグ時以外は必ず 0 にする。

Q. MOP (Measure of Performance) の計測方法は？

A. ファームの MOP_TEST フラグで計測コードをコンパイル時に選択する。例えば MOP_TEST=4 では楽器ノードが BEAT 受信 → 発音時刻の localMaster レンジを出力し, serial_logger.py で収集した後 analyze.py で集計する。通常ビルド (MOP_TEST=0) では計測コードは一切コンパイルされない。

Q. PC 側ログはファイルに保存されるか？

A. Processing の println() は stdout に出力され, processing-java --run で起動した場合はターミナルに表示される。ファイル保存は serial_logger.py 側で行う。Processing 画面内にもリングバッファ (200 行) でログパネルを表示している。

Q. ログからどんな問題を発見したか？

A. MOP5 検証で「SoftAP のマルチキャストが省電力バッファで 204.8ms 周期のバースト配送になっている」ことを発見した。これは BEAT の lookahead (当時 45ms) が構造的に不足していることを意味し, 220ms に引き上げた根拠になった。

Q. CPU 使用率の計測はどうしている？

A. MOP_TEST=8 で loop() 内の入力/ロジック/出力の 3 フェーズの micros() 差を毎ループ出力する。指揮者ノードの 3 フェーズ合計は 5ms 以内に収まっており、5ms 周期の IMU サンプリングに間に合う。

7 横断的な質問

Q. なぜこの構成（指揮者 + 楽器 + PC 分離）にしたか？

A. (1) UNO R4 WiFi 単体では加算合成のリアルタイム演算ができない ($44.1\text{ kHz} \times \text{倍音 } 20 \text{ 本} \times \text{同時発音数の浮動小数点演算量}$)。 (2) 指揮者は WiFi AP を起動する必要がある、ESP32-S3 が必要。 UNO R4 は STA のみ。 (3) 役割を分離することでファームウェアの複雑性を下げ、各ノードが単一責務で動く Embedded-Module-Architecture (EMA) を実現した。

Q. EMA (Embedded-Module-Architecture) とは何か？

A. loop() を入力 → ロジック → 出力の 3 フェーズに分け、各フェーズを IModule の配列で回す設計パターン。モジュール間は SystemData 構造体だけを介してデータを受け渡し、直接の関数呼び出しを禁止する。これにより IMU/WiFi/LED 等を独立にテスト・差し替えできる。

Q. 苦労した点は？

A. (1) SoftAP のマルチキャスト配送が DTIM バッファリングで 200ms 周期のバーストになる現象の特定と対策 (lookahead 220ms 化 + min フィルタ化)。 (2) 拍検出の閾値調整 (1.8→0.8→1.2g の試行錯誤、二重発火防止のゲート構造)。 (3) 4 声輪唱のサイクル窓ロジック (最終声部が 1 周を終えるまで先頭声部は次の周回を始めない制御)。

Q. 今後の展望は？

A. (1) ビーコン間隔の最適化で固定遅延 220ms を 70ms 程度に短縮。 (2) 楽器台数の拡張 (現行 5 台 → 6 台以上)。 (3) ゲームモードの拡充 (複数曲選択、難易度設定)。 (4) Processing から Web ベース (WebAudio) への移行検討。

Q. 費用・部品はいくらか？

A. 主要部品: XIAO ESP32-S3 Sense (～1200 円) × 1, Arduino UNO R4 WiFi (～3500 円) × 5, GY-521 (～300 円) × 1, ブレッドボード・ジャンパ線 (数百円)。 PC は大学の Mac を使用。 合計は約 2 万円前後。

Q. 生成 AI はどう使ったか？

A. 設計判断は著者が行い、AI (Claude) との対話を通じて共同でコードを書いた。 EMA のアーキテクチャ決定、Arduino の制約に照らしたレビュー、拍検出の閾値候補の提案などに活用。 最終的な判断と動作確認は人間が行い、誤りの責任は著者にある。

Q. 同期精度の MOP4 の実測値は？

A. 3 回実測 (延べ 560 拍 × 5 ノード) で中央値 7ms, 平均 10.8ms, 最大 65ms。 設計目標 20ms 以内を中央値で達成。 尾部の大きな値はループスツール (SoftAP バースト位相に起因) で 9.2% 程度発生。

Q. MOP5 (通信遅延) はどうか？

A. 受信遅刻率を当初の 45.4% から lookahead 220ms 化で 1.9–3.1% まで改善。「発音予約が成立する」ことを指標のスコープとして再定義した。